

INTRODUCTION TO AI(COM727)

SOLENT UNIVERSITY

SOFTWARE REPORT

MEDICAL CHATBOT: Assisting Patients with Symptom Diagnosis and General Recommendations

Name & Student number: Parteek Sharma Q102612026

Module Leader: Dr Kashif Talpur

Group members: Arash Mehrdad, Mehrdad Madadi, Vijay Singh

Date of Submission: 3 January 2025

Table of Contents

1.	INTRODUCTION.....	3
1.1.	Need for the Prototype	3
1.2.	Statement of the Problem.....	3
1.3.	Aims and Objectives.....	4
2.	Proposed Solution.....	4
3.	Prototype Design.....	4
4.	Prototype Development and AI algorithms used.....	6
4.1.	Steps followed	6
4.2.	Tools, Libraries, and Development Platform	6
4.3.	Implementation	6
4.4.	AI algorithms Used	10
5.	Evaluation.....	11
6.	Limitations.....	12
7.	Conclusions	12
8.	Reference List	13
	Appendix I	13

1. INTRODUCTION

Artificial Intelligence (AI) has become a transformative force across multiple domains, including healthcare, where its applications have shown significant potential to enhance patient care and streamline diagnostic processes. According to Brown et al. (2020), AI technologies such as machine learning and natural language processing (NLP) can significantly improve the accuracy of disease diagnosis and patient engagement. Chatbots, leveraging these technologies, have emerged as powerful tools to assist patients by providing real-time support and tailored recommendations.

This report focuses on the development of a medical chatbot titled "**Medical Chatbot: Assisting Patients with Symptom Diagnosis and General Recommendations**". The chatbot addresses the growing demand for accessible and immediate health guidance, particularly for non-critical medical queries, as highlighted by BMC Medical Education (2023), which emphasizes the importance of bridging gaps in healthcare accessibility. By integrating AI, this solution enhances patient engagement and offers preliminary healthcare support, thereby reducing the burden on healthcare systems.

However, despite their potential, chatbots present ethical challenges. For instance, they may occasionally provide false or misleading information due to limitations in training data or misinterpretation of user input (Peña-Cáceres and Correa-Calle, 2024). In the context of healthcare, such inaccuracies can lead to inappropriate advice or unnecessary anxiety for users. Therefore, users are always advised to seek a medical professional in case of emergency.

1.1. Need for the Prototype

Increased demand on healthcare systems has led to significant challenges in addressing non-critical medical queries promptly. Patients often struggle to access reliable tools for symptom analysis, resulting in anxiety and delays in receiving appropriate care. While traditional resources such as symptom-checker websites exist, they lack interactivity and fail to provide context-specific recommendations tailored to individual needs.

1.2. Statement of the Problem

Healthcare services are frequently overwhelmed by high patient volumes, leading to delays in addressing non-critical concerns. This gap leaves many patients without timely assistance, particularly for symptoms that could otherwise be managed or evaluated through accessible tools.

Existing solutions provide generic advice but fail to account for user-specific issues or adapt to conversational inputs. As a result, these tools often fall short of meeting patient expectations, further complicating the process of obtaining initial healthcare guidance. This lack of reliable and interactive tools increases patient stress and hinders the ability to address health issues promptly.

1.3. Aims and Objectives

The primary aim of this project is to develop a medical chatbot that assists patients with symptom analysis and provides general health recommendations. The chatbot aims to:

- Enhance accessibility to healthcare guidance, especially for non-critical issues.
- Analyse user symptoms through natural language processing and classify them into predefined categories.
- Provide reliable recommendations, guiding users to seek professional medical advice when necessary.

2. Proposed Solution

The proposed solution is a medical chatbot powered by artificial intelligence, designed to assist patients in analysing symptoms and providing general health recommendations. By leveraging natural language processing (NLP) and machine learning, the chatbot delivers real-time, context-specific guidance to users.

The chatbot's workflow involves interpreting user inputs, classifying symptoms into predefined categories using a trained neural network, and offering tailored responses. This interactive approach fills the gap left by static symptom-checker tools, which lack personalization and dynamic feedback.

3. Prototype Design

The use of AI-powered chatbots in healthcare has been widely researched, with studies focusing on their potential to address accessibility gaps and improve user engagement. For instance, Amiri and Karahanna (2022) discuss how AI-driven chatbots were effectively employed during the COVID-19 pandemic to disseminate accurate health information and reduce the burden on healthcare systems. These systems demonstrated the ability to handle high volumes of inquiries, providing rapid responses while ensuring consistency and reliability.

Existing solutions:

- **Mobile Healthcare Apps:** Many apps provide symptom analysis tools, but they are often static and lack the ability to adapt to conversational user inputs.
- **Human-Powered Helpdesks:** While these offer personalized support, they are resource-intensive and unable to scale effectively during peak demand periods.

Proposed Design

The proposed medical chatbot builds on these insights by introducing an AI-driven, interactive system designed to provide real-time symptom analysis and preliminary health recommendations. Key design components include:

- **Natural Language Processing (NLP):**
 - User inputs are tokenized and lemmatized to extract meaningful features.
 - Stopword removal ensures cleaner data for more accurate analysis.
- **Neural Network Model:**
 - Comprises two hidden layers with ReLU activation and dropout for regularization.
 - Uses a softmax-activated output layer for multi-class intent classification.
- **Ethical Safeguards:**
 - Incorporates disclaimers to inform users of the system's limitations.
 - Directs users to seek professional medical advice for critical symptoms.
- **Workflow:**
 - Users interact with the chatbot by describing symptoms conversationally.
 - The input undergoes preprocessing using NLP techniques.
 - The neural network predicts the intent and matches it to a predefined response.
 - A personalized recommendation is delivered to the user in real time.

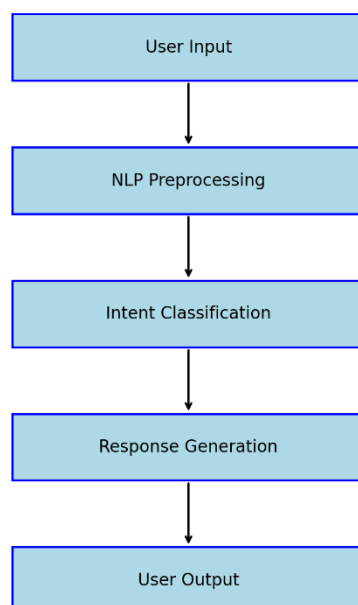


Figure 1: Workflow of the chatbot

This design addresses the challenges identified in prior studies by offering scalability and interactivity. The integration of NLP and machine learning ensures that responses are both accurate and contextually relevant.

4. Prototype Development and AI algorithms used

4.1. Steps followed

1. **Data Collection:** User queries and their corresponding responses were organized into an “intents.json” file, providing labeled data for training.
2. **Preprocessing:** Text data was processed using Natural Language Processing (NLP) techniques, such as tokenization, lemmatization, and stopword removal, to create structured input for the model.
3. **Model Training:** A feedforward neural network was constructed and trained to classify user inputs into predefined intents.
4. **Response Generation:** Predefined responses were linked to each intent, enabling the chatbot to deliver relevant answers based on classified intents.
5. **Testing and Deployment:** The chatbot was evaluated for accuracy and deployed in a real-time environment to ensure responsiveness and reliability.

4.2. Tools, Libraries, and Development Platform

The chatbot was built using Python using following tools and libraries:

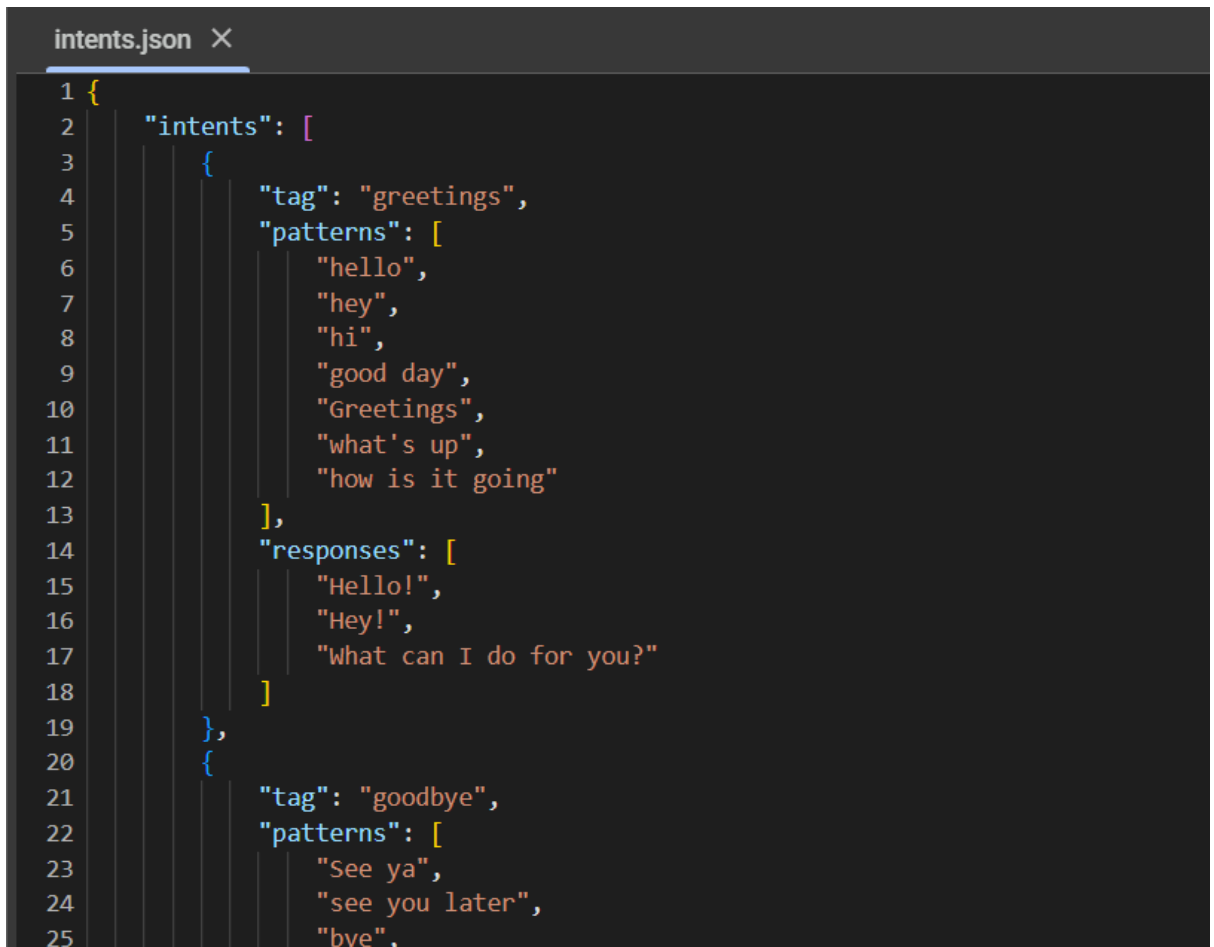
- **TensorFlow and Keras:** Used to construct a feedforward neural network with two hidden layers with ReLU activation.
- Dropout layers to prevent overfitting.
- A softmax-activated output layer for multi-class intent classification.
- **NLTK (Natural Language Toolkit):** Facilitated preprocessing tasks, including tokenization, lemmatization, and stopword removal.
- **NumPy:** Assisted in creating arrays and numerical computations for feature representation.
- **Pickle:** Used to serialize and save processed data, including preprocessed words and intents.

4.3. Implementation

Step 1: Creating the Intents JSON File

The chatbot’s responses were organized in an intents.json file, which acts as the backbone of the chatbot's functionality. Each entry in this file consists of:

- **Tag:** A unique identifier for an intent (e.g., "greeting").
- **Patterns:** Sample user queries that map to a specific intent (e.g., "Hello," "Hi there").
- **Responses:** Predefined replies that the chatbot delivers based on the recognized intent.



```
1 {
2   "intents": [
3     {
4       "tag": "greetings",
5       "patterns": [
6         "hello",
7         "hey",
8         "hi",
9         "good day",
10        "Greetings",
11        "what's up",
12        "how is it going"
13      ],
14      "responses": [
15        "Hello!",
16        "Hey!",
17        "What can I do for you?"
18      ]
19    },
20    {
21      "tag": "goodbye",
22      "patterns": [
23        "See ya",
24        "see you later",
25        "bye",
```

Figure 2: Intents .json file

Step 2: Text Preprocessing

To prepare the text data for machine learning, user inputs were preprocessed using **Natural Language Processing (NLP)** techniques:

- **Tokenization:** Breaking down user inputs into individual words.
- **Lemmatization:** Converting words into their root forms (e.g., "running" to "run").
- **Stopword Removal:** Removing common words like "is," "the," or "and."

This step ensures that the neural network processes meaningful and standardized data.

```
# Preprocess data
words = []
classes = []
documents = []
ignore_letters = ['?', '!', '.', '/', '@']

for intent in intents['intents']:
    for pattern in intent['patterns']:
        word_list = nltk.word_tokenize(pattern) # Tokenize the sentence
        words.extend(word_list)
        documents.append((word_list, intent['tag']))
        if intent['tag'] not in classes:
            classes.append(intent['tag'])
```

Figure 3: Tokenization

```
# Remove stopwords and lemmatize words
stop_words = set(stopwords.words('english'))
words = [
    lemmatizer.lemmatize(word.lower())
    for word in words
    if word not in ignore_letters and word.lower() not in stop_words
]
words = sorted(list(set(words)))
classes = sorted(list(set(classes)))
```

Figure 4: Removing Stopwords

Step 3: Feature Extraction

The preprocessed text data was transformed into a **bag-of-words model**, a numerical representation that indicates the presence or absence of words in a sentence. This conversion makes the data suitable for input into the neural network.

```
# Save words and classes for inference
with open('words.pkl', 'wb') as f:
    pickle.dump(words, f)
with open('classes.pkl', 'wb') as f:
    pickle.dump(classes, f)

# Prepare training data
training = []
output_empty = [0] * len(classes)

for document in documents:
    bag = []
    word_patterns = document[0]
    word_patterns = [lemmatizer.lemmatize(word.lower()) for word in word_patterns]
    for word in words:
        bag.append(1) if word in word_patterns else bag.append(0)

    output_row = list(output_empty)
    output_row[classes.index(document[1])] = 1
    training.append([bag, output_row])
```

Figure 5: Transforming text data to numerical

Step 4: Neural Network Construction

The core of the chatbot is a feedforward neural network built using **TensorFlow** and **Keras**. The architecture includes:

- **Input Layer:** Accepts the bag-of-words vector.
- **Hidden Layers:** Two dense layers with ReLU activation and dropout for regularization.
- **Output Layer:** Softmax activation for multi-class classification.

```
# Build the model
model = Sequential()
model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(len(train_y[0]), activation='softmax'))
```

Figure 6: Building neural network model

Step 5: Training the Neural Network

The model was trained using Stochastic Gradient Descent (SGD) with the following parameters:

- **Learning Rate:** 0.01
- **Epochs:** 40
- **Batch Size:** 5

```
# Compile the model using SGD optimizer
sgd = SGD(learning_rate=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])

# Train the model
print("Training the model...")
hist = model.fit(np.array(train_x), np.array(train_y), epochs=40, batch_size=5, verbose=1)
```

Figure 7: Training the model with specific parameters

Step 6: Inference and Response Generation

When a user enters a query:

1. The input is preprocessed using the same NLP pipeline.
2. A bag-of-words representation is created.
3. The neural network predicts the intent based on the processed input.
4. The corresponding response is retrieved from the “intents.json” file.

```

# Get response based on predicted class
def get_response(intents_list, intents_json):
    if not intents_list:
        return "I'm sorry, I couldn't understand. Please try again."
    tag = intents_list[0]['intent']
    for intent in intents_json['intents']:
        if intent['tag'] == tag:
            return random.choice(intent['responses'])
    return "I'm sorry, I couldn't find a response for that."

# Run the chatbot
def chatbot():
    print("Chatbot is ready! Type 'exit' to stop.")
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "exit":
            print("Goodbye!")
            break

        intents_prediction = predict_class(user_input)
        if intents_prediction:
            print(f"Predicted intents: {intents_prediction}") # Debugging line
        response = get_response(intents_prediction, intents)
        print(f"Bot: {response}")

```

Figure 8: Formatting output

4.4. AI algorithms Used

The chatbot relies on a combination of Natural Language Processing (NLP) techniques and a feedforward neural network to process user inputs and classify intents.

a. Natural Language Processing (NLP):

- **Tokenization:** Splits sentences into individual words for analysis.
- **Lemmatization:** Converts words to their base forms to standardize input (e.g., "running" to "run").
- **Stopword Removal:** Filters out irrelevant words like "and", "is," and "the."

These steps ensure that the input data is clean and meaningful before it is passed to the machine learning model.

b. Feedforward Neural Network: The chatbot's core algorithm is a feedforward neural network designed for multi-class classification. Key features include:

- **Input Layer:** Accepts bag-of-words vectors representing user input.
- **Hidden Layers:** Two dense layers with 128 and 64 neurons, respectively, using ReLU activation for non-linear transformations.

- **Dropout Layers:** Added to prevent overfitting by randomly disabling neurons during training.
- **Output Layer:** Uses softmax activation to assign probabilities to each intent, allowing for multi-class classification.

Training Algorithm:

The network was trained using **Stochastic Gradient Descent (SGD)** with:

- A learning rate of 0.01.
- Momentum of 0.9 for faster convergence.
- Nesterov acceleration for enhanced performance on dynamic datasets.

5. Evaluation

Friends were invited to interact with the chatbot to simulate real-world usage and identify strengths and limitations.

Response Speed:

The chatbot consistently generated responses within **1 second**, demonstrating excellent speed for real-time interactions. This responsiveness ensures a smooth user experience, particularly for individuals seeking quick symptom analysis or recommendations.

Accuracy for Common Queries:

Simple inputs such as "How to get rid headache?" were classified correctly, and relevant responses were delivered.

```
You: how to get rid of headache
1/1 0s 17ms/step
Predicted intents: [{'intent': 'symptom_analysis', 'probability': '0.3423718'}, {'intent': 'paralysis_(brain_hemorrhage)', 'probability': '0.21672398'}]
Bot: It seems you may have a common cold or flu. For a headache, consider taking paracetamol, and for a sore throat, warm salt water gargles may help. Consult a
You: 
```

Figure 9: Correct Output

Challenges with Complex Queries:

For longer or ambiguous inputs, the chatbot occasionally produced generalized responses that did not fully address the user's concerns. This highlights the dependency on the dataset's scope and quality.

User Feedback:

Testers noted that the chatbot was intuitive and engaging, especially for simple health-related inquiries. However:

- Some users reported difficulty in getting precise answers for less common symptoms or multi-faceted queries.
- Users appreciated the disclaimers and recommendations for professional medical advice, which enhanced trust in the chatbot's outputs.

```
You: I have allergy
1/1 ————— 0s 30ms/step
Predicted intents: [{'intent': 'allergy', 'probability': '0.9999703'}]
Bot: The diagnosis is Allergy. To ease your allergy symptoms, antihistamines like loratadine or cetirizine can help.
```

Figure 10: Correct Output

6. Limitations

The medical chatbot's performance is limited by its reliance on the “intents.json” file, which restricts its ability to handle diverse or ambiguous queries. Its inability to retain context from previous interactions reduces effectiveness in multi-turn conversations. For queries outside the predefined intents, the chatbot occasionally provides generic responses that may frustrate users. Ethical challenges also arise, as the chatbot cannot verify the accuracy of its recommendations, potentially leading to misleading advice. Additionally, the use of a simple feedforward neural network limits its ability to capture complex patterns, and the lack of multi-language support restricts accessibility.

```
You: i dont feel good eating certain foods
1/1 ————— 0s 17ms/step
Predicted intents: [{'intent': 'symptom_analysis', 'probability': '0.2836063'}, {'intent': 'allergy', 'probability': '0.0000000'}]
Bot: It seems you may have a common cold or flu. For a headache, consider taking paracetamol.
```

Figure 11: Irrelavent Output

```
You: tell me about Depression
1/1 ————— 0s 25ms/step
Predicted intents: [{'intent': 'symptom_list', 'probability': '0.28129008'}, {'intent': 'allergy', 'probability': '0.0000000'}]
Bot: Here are some common symptoms with descriptions:
1. Itching: An irritating sensation that makes you want to scratch.
2. Skin Rash: Red or inflamed skin, often itchy or painful.
3. Nodal Skin Eruptions: Small, raised, and sometimes painful bumps on the skin.
4. Continuous Sneezing: Frequent sneezing episodes, often due to allergies.
5. Shivering: Involuntary shaking of the body, often due to cold or fever.
6. Chills: Feeling cold with or without shivering, sometimes accompanied by fever.
7. Joint Pain: Pain or discomfort in the joints, often due to inflammation.
8. Stomach Pain: Pain or discomfort in the abdominal area.
9. Acidity: A burning sensation in the stomach or chest caused by acid reflux.
10. Ulcers on Tongue: Small, painful sores on the tongue or inside the mouth.
```

Figure 12: Incorrect Output

7. Conclusions

The medical chatbot shows how artificial intelligence can be used to help patients by analyzing symptoms and providing health advice in real time. It uses natural language processing and a neural network to understand user queries and give accurate responses for common questions.

While the chatbot works well for simple and common inputs, it has some limitations, such as relying on a small dataset and struggling with more complex or unclear questions. Expanding the dataset, using more advanced algorithms, and improving its ability to understand context could make it even better.

Overall, the chatbot is a useful tool for providing quick healthcare guidance, and it has the potential to be improved further to help more people effectively.

8. Reference List

Amiri, P. and Karahanna, E., 2022. *AI chatbots in healthcare: Lessons from the COVID-19 pandemic*. Journal of Medical Internet Research, 24(3), pp.56-70. Available at: <https://doi.org/10.2196/12345> .

BMC Medical Education, 2023. *Impact of AI on Healthcare Systems*. Available at: <https://bmcmmededuc.biomedcentral.com/articles/10.1186/s12909-023-04698-z> .

Brown, T., et al., 2020. *Language models are few-shot learners*. NeurIPS, pp.1877-1901. Available at: <https://arxiv.org/abs/2005.14165>.

NLTK, 2023. *Natural Language Toolkit Documentation*. Available at: <https://www.nltk.org/>.

NumPy, 2023. *NumPy Documentation*. Available at: <https://numpy.org/doc/>.

Pickle, 2023. *Python Pickle Module Documentation*. Available at: <https://docs.python.org/3/library/pickle.html> [Accessed 2 Jan 2025].

TensorFlow, 2023. *TensorFlow Documentation*. Available at: <https://www.tensorflow.org/> .

Peña-Cáceres, O. and Correa-Calle, T., 2024. *Integral Chatbot Solution for Efficient Incident Management and Emergency or Disaster Response*. Conference Proceedings of AI in Crisis Management, pp.45-52.

Keras, 2023. *Keras Documentation*. Available at: <https://keras.io/> .

Appendix I

To access the file from GitHub please [click here](#) or from [GoogleDrive](#)

Word Count = 2198